

Intent Routing

Hermes Agent Intent Routing

Table of Contents

- 1. [I. Overall Architecture Overview](#)
- 2. [II. Command Routing — The Core Intent Routing Layer](#)
 - [2.1 Command Registry](#)
 - [2.2 Command Parsing and Dispatch](#)
 - [2.3 ACP Adapter Command Routing](#)
 - [2.4 Skill Command Routing](#)
 - [2.5 Quick Commands](#)
- 3. [III. Model Routing](#)
 - [3.1 Overview](#)
 - [3.2 Keyword Heuristics](#)
 - [3.3 Routing Decision Logic](#)
 - [3.4 Routing Integration Entry Point](#)
- 4. [IV. Provider Routing](#)
 - [4.1 OpenRouter Provider Preference Routing](#)
 - [4.2 Runtime Provider Resolution](#)
- 5. [V. Message Delivery Routing](#)
 - [5.1 DeliveryRouter](#)
 - [5.2 Delivery Target Parsing Rules](#)
- 6. [VI. Sub-agent Delegation Routing](#)
- 7. [VII. Platform Adapter Routing](#)
 - [7.1 Architecture](#)
 - [7.2 Platform Adapter Registration](#)
- 8. [VIII. Hook System](#)
- 9. [IX. CLI-side Routing](#)
- 10. [X. Summary](#)
 - [Key Design Principles](#)

I. Overall Architecture Overview

Hermes Agent's **Intent Routing** is not a standalone module or an ML-driven intent recognition system, but rather a **multi-layered, multiplexed hierarchical decision mechanism**. It makes routing decisions at different layers after user messages enter the system:

Layer	Routing Target	Driven By	File Location
Command Routing	Built-in commands starting with / vs Skill commands vs Quick Commands	String prefix matching + Command registry	gateway/run.py , acp_adapter/server.py
Model Routing	Choose strong model (default) or cheap model	Keyword heuristic rules	agent/smart_model_routing.py

Layer	Routing Target	Driven By	File Location
Message Routing	Which Messaging Platform Adapter to deliver messages to	Session Source tracking	gateway/delivery.py
Tool Routing	LLM decides which tool to call	LLM's own decision	tools/registry.py
Sub-agent Routing	Delegate tasks to which sub-agent/provider	Config-driven	tools/delegate_tool.py
Platform Routing	Which messaging platform an event comes from	Event Source parsing	gateway/run.py

II. Command Routing (Command Routing) — The Core Intent Routing Layer

2.1 Command Registry

File: hermes_cli/commands.py

This is the "Single Source of Truth" for the entire command routing system.

```
# Data structure
@dataclass(frozen=True)
class CommandDef:
    name: str # Canonical name without slash: "background"
    description: str # Human-readable description
    category: str # Category: "Session", "Configuration", etc.
    aliases: tuple[str, ...] = () # Aliases: ("bg",)
    args_hint: str = "" # Argument hint: "<prompt>", "[name]"
    subcommands: tuple[str, ...] = () # Subcommands for tab completion
    cli_only: bool = False # Only available in CLI
    gateway_only: bool = False # Only available in Gateway/messaging platforms
    gateway_config_gate: str | None = None # Configuration gate
```

All commands are registered in the `COMMAND_REGISTRY` list as `CommandDef` entries. Key built-in command categories:

Session Commands:

- `/new`, `/reset` — New session
- `/retry` — Retry last message
- `/undo` — Remove last exchange
- `/compress` — Manually compress conversation context
- `/background` (`/bg`) — Run prompt asynchronously in background
- `/btw` — Ephemeral side question (no tools, no persistence)
- `/queue` (`/q`) — Queue a message (does not interrupt current run)
- `/rollback` — Restore filesystem checkpoint
- `/stop` — Terminate all background processes
- `/approve`, `/deny` — Approve/deny dangerous commands

Configuration Commands:

- `/reasoning` — Manage reasoning effort and display
- `/verbose` — Toggle tool progress display mode
- `/yolo` — Skip all dangerous command approvals
- `/personality` — Set predefined persona

- `/provider` — Show current provider

Tools & Skills Commands:

- `/skills` — Search/install/view skills
- `/cron` — Manage scheduled tasks
- `/reload-mcp` — Reload MCP servers
- `/browser` — Connect browser tool
- `/plugins` — List installed plugins

Info Commands:

- `/commands` — Browse all commands with pagination
- `/help` — Show help
- `/usage` — Show token usage
- `/insights` — Usage insight analysis
- `/platforms` — Gateway platform status

2.2 Command Parsing and Dispatch (Gateway Layer)

File: `gateway/run.py` — `_handle_message()` method (~line 1678)

This is the entry method where Gateway receives events from various messaging platforms (telegram, discord, slack, etc.). The message processing pipeline is as follows:



2.2.1 MessageEvent.get_command() — Command Extraction

File: gateway/platforms/base.py — MessageEvent class

```
class MessageEvent:
    text: str
    message_type: MessageType = MessageType.TEXT
    source: SessionSource = None

    def is_command(self) -> bool:
        return self.text.startswith("/")

    def get_command(self) -> Optional[str]:
        """Extract command name from messages starting with `/`."""
        if not self.is_command():
            return None
        parts = self.text.split(maxsplit=1)
        raw = parts[0][1:].lower() if parts else None
        if raw and "@" in raw:
            raw = raw.split("@", 1)[0] # Handle Telegram @mention format
        return raw

    def get_command_args(self) -> str:
        """Get arguments after the command."""
        if not self.is_command():
            return self.text
        parts = self.text.split(maxsplit=1)
        return parts[1] if len(parts) > 1 else ""
```

Routing Decision Point: This is a pure string operation, determining if it's a command by checking `text.startswith("/")`. No ML or complex NLP is required.

Note

Command Routing is Essentially a Deterministic Finite Automaton (DFA)

Negative Routing Pattern (or "LLM as final fallback"): Hermes's command dispatch embodies a design principle opposite to most intent classification systems: **only "unrecognized" commands flow to the LLM**

2.2.2 Command Alias Resolution

File: hermes_cli/commands.py — `resolve_command()` function

```
def resolve_command(name: str) -> CommandDef | None:
    """Resolve command name or alias to CommandDef."""
    return _COMMAND_LOOKUP.get(name.lower().lstrip("/"))

# Build lookup table
_COMMAND_LOOKUP: dict[str, CommandDef] = _build_command_lookup()
# Contains both command names and all aliases (e.g., "bg" → CommandDef("background", ...))
```

Key: In `gateway/run.py`, aliases are resolved to canonical names before dispatch:

```
_cmd_def = _resolve_cmd(command) if command else None
canonical = _cmd_def.name if _cmd_def else command
# All subsequent if checks are based on canonical names
```

2.2.3 GATEWAY_KNOWN_COMMANDS — Gateway Recognizable Command Set

```
GATEWAY_KNOWN_COMMANDS: frozenset[str] = frozenset(
    name
    for cmd in COMMAND_REGISTRY
    if not cmd.cli_only or cmd.gateway_config_gate # Include all gateway-side commands
    for name in (cmd.name, *cmd.aliases)
)
```

Usage:

1. **Telegram BotCommands Menu** — Telegram only supports `/command` format menus
2. **Slack Subcommand Mapping** — `slack_subcommand_map()` generates slash handler mappings
3. **Command Completion** — `SlashCommandCompleter` provides tab completion

2.3 ACP Adapter Command Routing (Headless Scenario)

File: `acp_adapter/server.py`

The ACP (Agent Client Protocol) adapter handles messages from editors (like Claude Code) in headless scenarios:

```
class HermesACPAgent(acp.Agent):
    # Intercept slash commands (no LLM call needed)
    if user_text.startswith("/"):
        response_text = self._handle_slash_command(user_text, state)
        if response_text is not None:
            # Return text response directly, skip LLM
            return PromptResponse(stop_reason="end_turn")

    # Unrecognized command → Send to LLM (user may have typed /something as text input)
```

Built-in headless command list:

```
_SLASH_COMMANDS = {
    "help": "Show available commands",
    "model": "Show or change current model",
    "tools": "List available tools",
    "context": "Show conversation context info",
    "reset": "Clear conversation history",
    "compact": "Compress conversation context",
    "version": "Show Hermes version",
}
```

Note

The headless scenario refers to a server environment "without GUI, without audio devices, without interactive terminals". Hermes Agent ensures normal operation (rather than crashing) in such situations through environment detection, lazy loading, safe wrappers, and other degradation strategies.

2.4 Skill Command Routing (Dynamic Commands)

File: `agent/skill_commands.py` — `scan_skill_commands()`

Skills are stored as `SKILL.md` files in the `~/.hermes/skills/` directory, and each skill automatically registers as a slash command:

```
def scan_skill_commands() -> Dict[str, Dict[str, Any]]:
    """Scan skills/ directory, return /command-name → skill info mapping."""
    # Traverse ~/.hermes/skills/**/*.md
    # Parse name and description from frontmatter
    # Command name = name.lower().replace(' ', '-')
    # Register to _skill_commands global dict
```

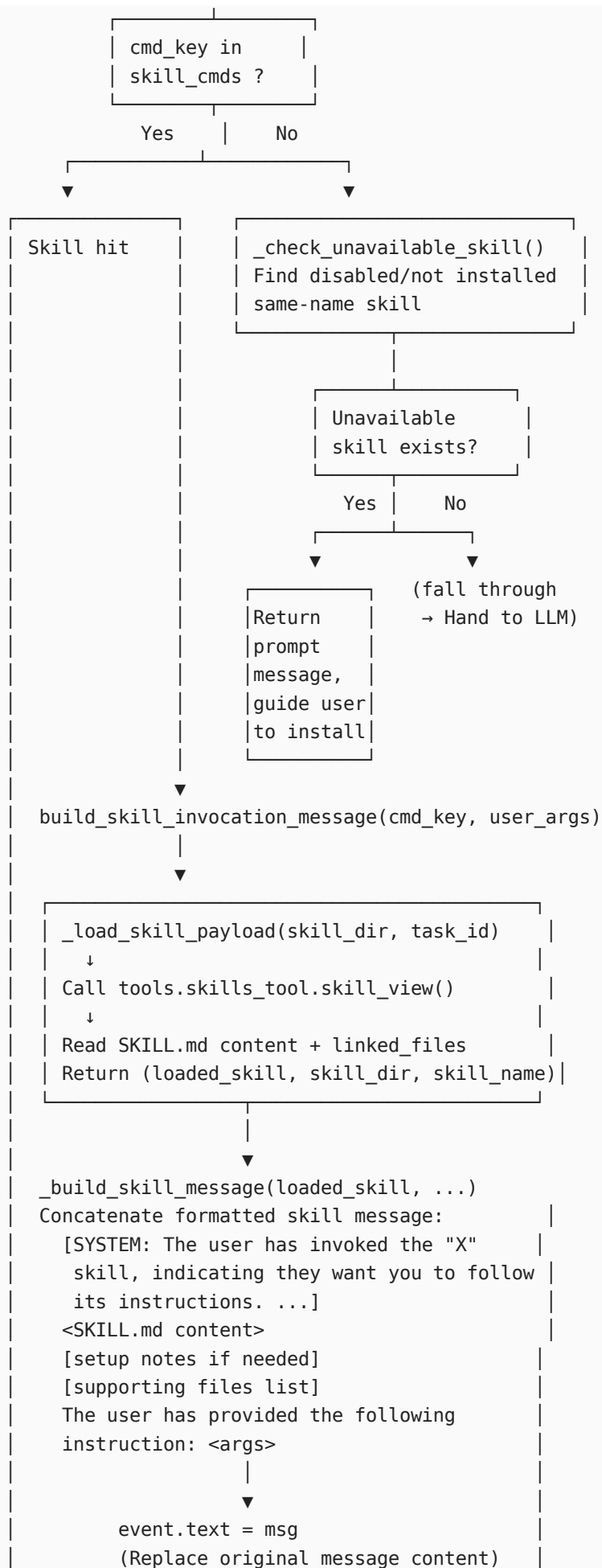
Skill Command Dispatch Flow (gateway/run.py lines 2028-2048):

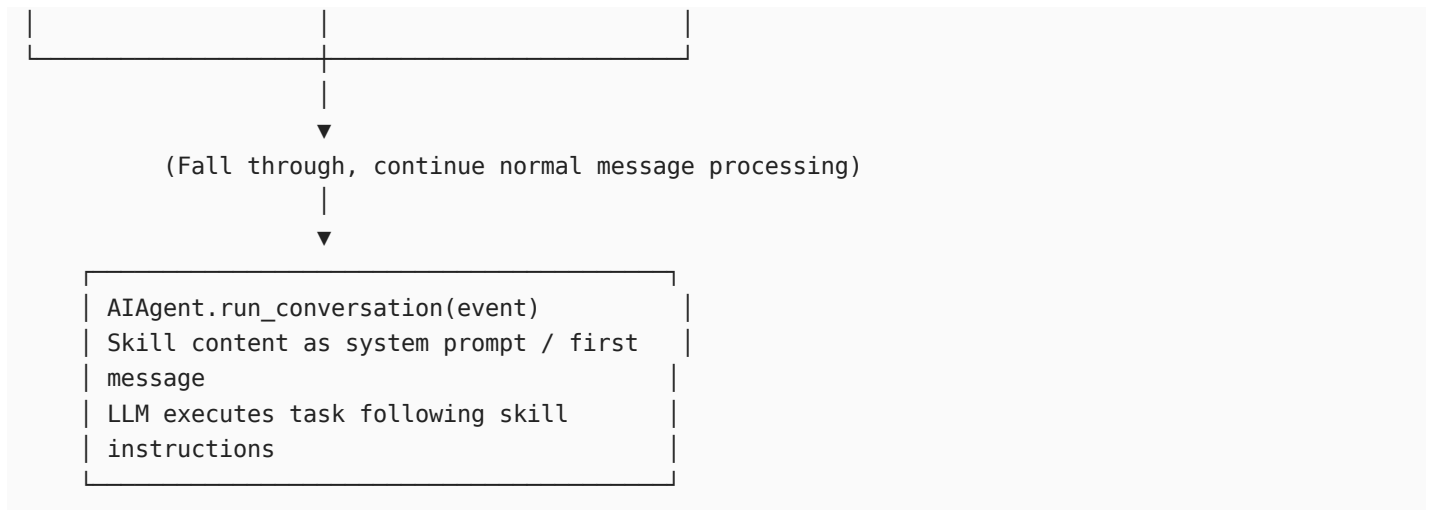
Skills are stored as `SKILL.md` files in the `~/.hermes/skills/` directory, scanned and registered by `scan_skill_commands()` at system startup. Command names are derived from the `name` field in frontmatter (spaces replaced with `-`, converted to lowercase).

```
if command:
    from agent.skill_commands import get_skill_commands, build_skill_invocation_message
    skill_cmds = get_skill_commands() # Returns {"skill-name": {...}, ...}
    cmd_key = f"/{command}"
    if cmd_key in skill_cmds:          # Skill command hit
        user_instruction = event.get_command_args().strip()
        msg = build_skill_invocation_message(cmd_key, user_instruction, task_id=_quick_key)
        if msg:
            event.text = msg           # Replace message content with skill prompt
            # Continue fall through → Hand to LLM (skill content as prompt)
    else:
        # Try to match "known but disabled/not installed" skill → Give actionable hint
        _unavail_msg = _check_unavailable_skill(command)
        if _unavail_msg:
            return _unavail_msg
```

Below is the complete Skill command dispatch flowchart:







2.5 Quick Commands (User-defined Quick Commands)

File: gateway/run.py lines 1971-2010

Users can define quick commands in `config.yaml` that do not require LLM calls:

```
quick_commands:
  hello:
    type: exec
    command: "echo Hello from Hermes!"
  greet:
    type: alias
    target: /background
```

- `type: exec` → Execute shell command directly, return output
- `type: alias` → Alias to another command, continue dispatch flow

III. Model Routing (Model Routing) — Smart Cheap vs. Strong

3.1 Overview

It decides whether to use the **cheap model** or the **default strong model** at the single-turn conversation level based on **surface-level features** (length, keywords, etc.) of user input.

File: agent/smart_model_routing.py

Note

Per-turn Independent Decision vs. Cross-turn State — Motivation for Abandoning MDP Modeling

`choose_cheap_model_route()` makes decisions independently for each message ("stateless per-turn"), without introducing cross-turn conversation hidden states (e.g., "last turn used cheap model and user didn't complain" → continue with cheap).

From an algorithmic perspective, cross-turn state modeling can capture users' "implicit feedback" (e.g., whether user immediately follows up or complains) to dynamically adjust routing, but the cost is: (1) significantly increased implementation complexity; (2) additional instrumentation and feedback collection mechanisms needed; (3) state space explosion requires online learning or rule engine maintenance.

The current independent decision model is equivalent to assuming each turn's message complexity is unrelated to

other turns — this is certainly oversimplified, but in practice, as long as most users' simple questions can indeed be resolved in a single turn, the error introduced by this simplification is acceptable.

3.2 Keyword Heuristics (Simple Intent Detection)

```
_COMPLEX_KEYWORDS = {
    "debug", "debugging", "implement", "refactor", "patch",
    "traceback", "stacktrace", "exception", "error",
    "analyze", "analysis", "investigate",
    "architecture", "design", "compare", "benchmark", "optimize",
    "review", "terminal", "shell", "tool", "tools",
    "pytest", "test", "tests",
    "plan", "planning", "delegate", "subagent",
    "cron", "docker", "kubernetes",
}
```

Any hit triggers the strong model; no hit and length constraints satisfied means use the cheap model. This set is manually maintained and fully explainable, but coverage is limited by maintainer experience (e.g., includes "debug" but not "troubleshoot").

3.3 Routing Decision Logic

```
def choose_cheap_model_route(user_message: str, routing_config) -> Optional[Dict]:
    """If message looks simple, return configured cheap model; otherwise return None (use primary
    model)."""

    # Decision conditions (ALL must be satisfied to use cheap model):
    if len(text) > max_chars (default 160): return None
    if word_count > max_words (default 28): return None
    if newline_count > 1: return None
    if "`" in text or "`" in text: return None # Code
    if URL_RE.search(text): return None # URL
    if words & _COMPLEX_KEYWORDS: return None # Complex keywords

    # None of the above triggered → Use cheap model
    return cheap_model_config
```

Characteristics:

- **Conservative Strategy:** Prefer using strong model by mistake, rather than failing on complex tasks with cheap model
- **Zero-shot:** Completely rule-based, no training
- **Configurable:** Configure `cheap_model` and threshold parameters in `config.yaml`

Note

The Essence of Conservative Strategy is an Extreme Choice in Precision-Recall Tradeoff

The costs of the two types of errors are severely asymmetric:

- **Miss (complex→cheap):** Task failure, tool call errors, user needs to retry → High cost
- **False alarm (simple→strong model):** Extra cost → Low cost

Therefore, thresholds are set very tight (`max_chars=160` , `max_words=28` , reject all code), equivalent to the optimal conservative boundary in Bayesian decision theory with asymmetric loss.

Note

Survivorship Bias Trap in Keyword Set

Words in the set = words "observed to appear in complex tasks"; words not in it may:

1. Actually strongly correlate with simple tasks
2. Also appear in complex tasks but weren't sampled
3. Be ambiguous (e.g., "test": unit test request is complex, casual conversation is simple)

Recommendation: Run offline analysis periodically, calculate $P(\text{simple}|\text{keyword})$, use Precision/Recall curves to evaluate word additions/deletions.

3.4 Routing Integration Entry Point

File: `gateway/run.py` lines 779-791

```
def _resolve_turn_agent_config(self, user_message: str, model: str, runtime_kwargs: dict) -> dict:
    from agent.smart_model_routing import resolve_turn_route
    primary = {"model": model, ...}
    return resolve_turn_route(user_message, self._smart_model_routing, primary)
```

Called once per message turn for independent routing decisions.

IV. Provider Routing (Provider Routing)

4.1 OpenRouter Provider Preference Routing

File: `gateway/run.py` lines 996-1007, `hermes_cli/runtime_provider.py`

Loaded from `config.yaml`:

```
provider_routing:
    # Specify preferred OpenRouter provider for specific models
    # E.g., force certain models through specific provider for better pricing/availability
```

4.2 Runtime Provider Resolution

File: `hermes_cli/runtime_provider.py` — `resolve_runtime_provider()`

Supports unified resolution across multiple providers:

- `openrouter` — OpenRouter aggregation platform
- `nous` — Nous Research
- `zai` — Zai platform
- `kim-coding` — Kimi Coding
- `minimax` — MiniMax
- `openai` — OpenAI Direct
- `custom` — User-defined provider

V. Message Delivery Routing (Delivery Routing)

5.1 DeliveryRouter

File: gateway/delivery.py — DeliveryRouter class

Delivery target resolution for cron task outputs and Agent responses:

```
@dataclass
class DeliveryTarget:
    platform: Platform          # telegram, discord, slack, feishu, ...
    chat_id: Optional[str] = None # None → Use home channel
    thread_id: Optional[str] = None
    is_origin: bool = False      # Whether to respond to origin
    is_explicit: bool = False     # Whether explicitly specified

    @classmethod
    def parse(cls, target: str, origin: Optional[SessionSource]) -> "DeliveryTarget":
        """Parse delivery target string."""
        # "origin" → Respond to origin
        # "local" → Local file
        # "telegram" → Telegram home channel
        # "telegram:123456789" → Specified chat
```

PS: **Cron Jobs:** Workflows or scripts that automatically trigger periodically according to preset time expressions (cron expressions). For example, "fetch news summary every morning at 8am".

Note

Triple Key for Multi-target Deduplication — Business Invariance vs. Implementation Simplicity Tradeoff

Deduplication key = (platform, chat_id, thread_id) triple, is_origin not included in key:

- ["telegram", "origin"] and origin = telegram → Deduplicate, send once ✓
- ["telegram", "origin"] and origin = discord → Keep both ✓

The string "origin" is normalized to origin.platform in to_string(), equivalent to explicit platform — this is **semantic normalization**: different strings representing the same target are unified before delivery, no need to send duplicates due to different origin markers.

Note

Hop-by-hop Appending vs. Global Deduplication — Write Order Dependency Risk

always_log_local=True + "local" already in deliver → local appended twice (once in loop, once at end).

Implementation prevents duplicate delivery through deduplication key, but the double append logic itself easily causes confusion.

Recommendation: Use seen_platforms to handle always_log_local uniformly in the loop, eliminating the implicit behavior of appending at the end.

Note

Platform Message Truncation is Defensive Programming, Not Routing Decision

MAX_PLATFORM_OUTPUT = 4000 is proactive trimming based on downstream platform API constraints:

- **Positioning:** `DeliveryRouter`'s responsibility is "where to send", not "how big it can send"
- **Strategy:** Different platforms have different rate limits, only known at runtime, defensive trimming is the most robust degradation
- **Guarantee:** Pointer left at truncation point (`full output saved to {path}`), message content not lost

In line with "Conservative Strategy" (prefer not to use cheap model): better to lose the tail than have the entire message delivery fail.

5.2 Delivery Target Parsing Rules

```
# Supported target formats:
# "origin"      → Respond to request origin messaging platform
# "local"       → Save to local file only
# "telegram"    → Telegram home channel
# "telegram:123456" → Specified Telegram chat
# ["local", "telegram"] → Multiple targets
```

Cron Job's `deliver` field uses this mechanism.

VI. Sub-agent Delegation Routing (Delegation Routing)

File: `tools/delegate_tool.py`

When the Agent uses the `delegate_task` tool:

```
def resolve_delegation_provider(configured_provider: str, ...) -> Dict:
    """Resolve provider and credentials used by sub-agent."""
    # Support configuring different providers for sub-agents
    # E.g., parent agent uses Nous, sub-agent uses OpenRouter + cheap model
```

The `delegation` section in `config.yaml` can configure:

- Provider used by sub-agent
- Model used by sub-agent
- Toolset available to sub-agent

VII. Platform Adapter Routing (Platform Adapter Routing)

7.1 Architecture

File: `gateway/platforms/` directory

Each message identifies the source platform via `SessionSource`:

```
@dataclass
class SessionSource:
    platform: Platform      # telegram, discord, slack, feishu, ...
    chat_id: str
    user_id: str
    thread_id: Optional[str] = None
    chat_type: str = "dm"   # "dm" | "group"
```

7.2 Platform Adapter Registration

File: gateway/run.py

```
class GatewayRunner:
    adapters: Dict[Platform, BasePlatformAdapter] = {}

    # Register all enabled platform adapters at startup
    # Dynamic add/remove during runtime (hot-swapping)
    # Message → _handle_message() → Check source.platform → Corresponding adapter
```

VIII. Hook System — Extensible Event Routing

File: gateway/hooks.py — HookRegistry class

Event-driven extensible routing:

```
# Supported event types:
gateway:startup      # Gateway process startup
session:start        # New session created (first message)
session:end          # Session ended (/new or /reset)
session:reset        # Session reset completed
agent:start          # Agent starts processing message
agent:step           # Each turn in tool call loop
agent:end            # Agent processing completed
command:*            # Any slash command executed (wildcard)
```

Wildcard Matching: Handlers can register "command:*" to listen to all command events.

IX. CLI-side Routing

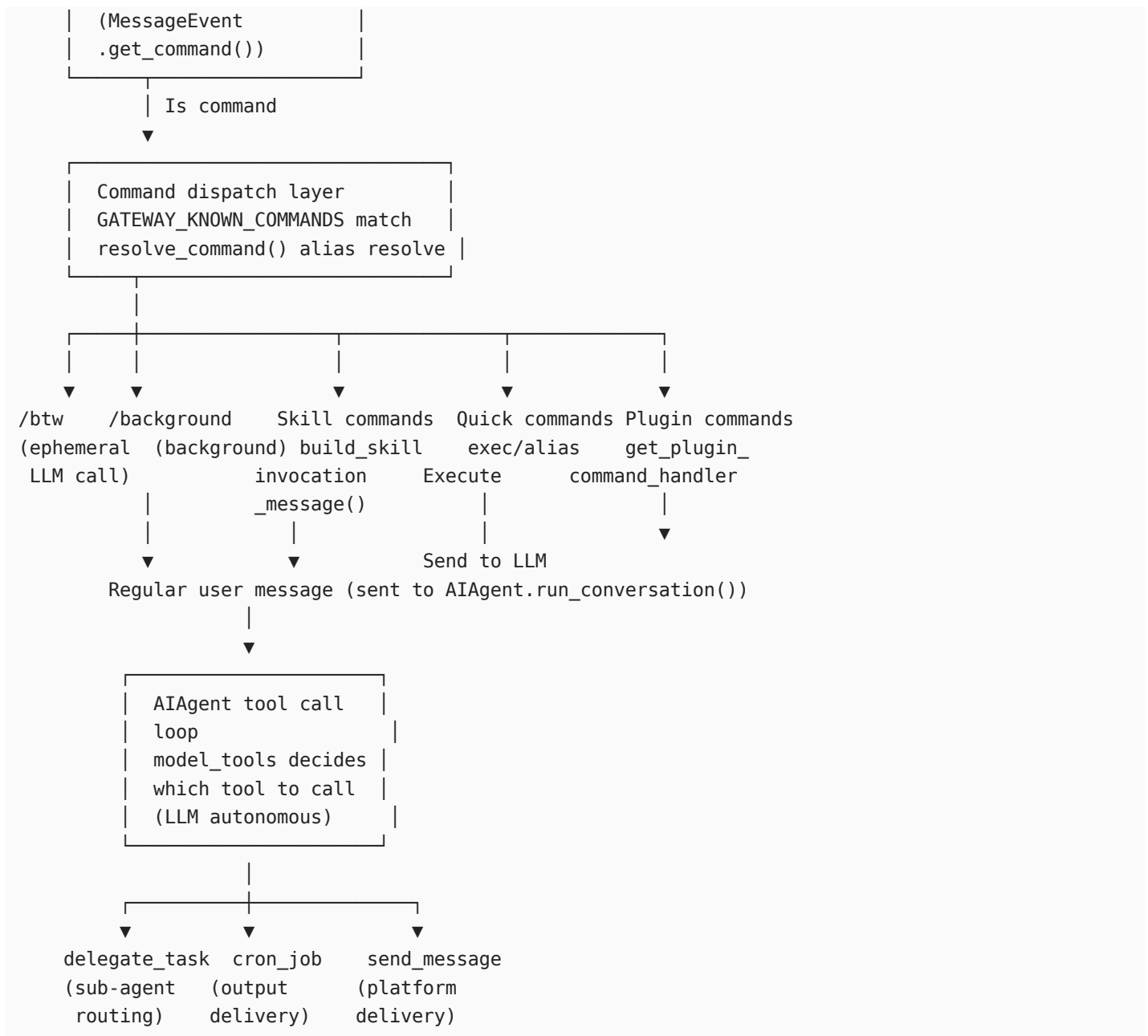
File: cli.py (Main CLI REPL)

CLI-side routing is relatively simple, providing command completion via prompt_toolkit's SlashCommandCompleter :

```
# CLI input → Starts with `/` → Use command routing
# Otherwise → Send to LLM normally
```

X. Summary: Hermes Intent Routing Overview





Key Design Principles

1. **Command routing takes priority over LLM:** All messages starting with `/` go through command routing first; only unmatched commands go to LLM
2. **Unified alias resolution:** All aliases are normalized to canonical names before dispatch
3. **Hierarchical decision-making:** Model routing at message layer, delivery routing at output layer, platform routing at access layer
4. **No ML intent recognition:** All routing decisions are based on string matching or keyword rules, no trained models
5. **Extensibility:** Support dynamic extension through Hook system, `register_plugin_command()`, and Skill mechanism
6. **Zero ambiguity handling:** Aliases point to only one canonical command, no polysemy

Note

The Hidden Philosophy of Hermes Routing: Determinism Over Flexibility, Latency Over Capability

Looking at this report holistically, an algorithm philosophy permeating the entire system emerges: Hermes's routing system is essentially a **rule-first, LLM-as-fallback hierarchical decision tree**, not the currently popular "LLM as the brain for unified scheduling" approach. This has several deep implications algorithmically:

- **Zero-hallucination routing:** Rule routing never "thinks wrong" — given the same input, it guarantees the same output. LLM routing (if any) is inherently non-deterministic; the same input may yield different routing decisions due to temperature or context. In mission-critical systems, this is a non-negligible reliability difference.
- **Active latency budget management:** Each additional LLM call layer adds 100-500ms to p99 latency. Typical rule routing latency ranges from microseconds (dict lookup) to milliseconds (I/O), a gap of 2-3 orders of magnitude. In multi-tenant Gateway scenarios, rule-first directly determines the system's capacity ceiling.
- **Cost predictability:** LLM calls have token costs that grow non-linearly with traffic (long conversation context expansion). Rule routing costs are fixed infrastructure costs, easier for capacity planning and budget control.

This design is reasonable and efficient under the current constraints of a fixed command set (hundreds of slash commands) and fixed platform set (dozens of messaging platforms). However, if future needs require supporting **user-defined natural language intents** (e.g., "help me analyze recent logs"), this system will face fundamental scaling challenges: keyword rules will explode combinatorially, $O(n)$ rule maintenance becomes unsustainable, and introducing lightweight embedding models (embedding-based classifier) for natural language intent matching will be almost inevitable

Note

Comparison with Industry Mainstream Approaches: Rule System vs. LLM Routing Tradeoffs

Core Insight: The choice of routing strategy depends on whether the input space is closed or open.

Input Space	Optimal Solution	Reason
Closed (slash commands, finite enumerable)	Rule routing	Provable correctness + $O(1)$ complexity
Open (natural language intents)	LLM routing	Cannot enumerate, must understand semantics

Hermes's slash command space is closed, and the designers made the correct technical choice. LLM-native routing (LangChain Agent Router, OpenAI Function Calling) is suitable for open-domain scenarios, but is "using a sledgehammer to crack a nut" in closed spaces.